Feedforward Neural Network

Tugas Besar 1 IF3270 Pembelajaran Mesin

Disusun oleh Tim:

Bigmo Masterclass

Nama	NIM
Orvin Andika Ikhsan A	13523017
Fajar Kurniawan	13523027
Adhimas Aryo Bimo	13523052

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2025

Daftar Isi

Daftar Isi 2
Daftar Gambar dan Tabel 3
01 - Deskripsi Persoalan 4
02- Pembahasan 5
1. Struktur Kode 5
2. Penjelasan Hasil Implementasi 6
2.1. Diagram Kelas 6
2.2. Deskripsi Kelas 7
2.3. Penjelasan Implementasi 9
2.4. Forward Propagation 15
2.5. Backward Propagation 15
3. Hasil Pengujian 16
3.1. Pengaruh Depth dan Width 16
3.2. Pengaruh Fungsi Aktivasi 17
3.3. Pengaruh Learning Rate 19
3.4. Pengaruh Regularisasi 21
3.5. Pengaruh Inisialisasi Bobot 23
3.6. Pengaruh Normalisasi RMSNorm 24
3.7. Kecepatan Konvergensi Adam vs Gradient Descent 26
3.8. Perbandingan dengan Library Scikit-Learn 27
03- Kesimpulan dan Saran 28
Kesimpulan 28
Saran 28
04 - Pembagian Tugas 30
05 - Referensi 30
06 - Lampiran 31

Daftar Gambar dan Tabel

Gambar 2.1 Diagram Kelas..... 6
Tabel 2.1 Deskripsi Kelas.....9
Gambar 3.1.1 Grafik Loss Pengujian Depth dan Width..... 16
Gambar 3.1.2 Hasil Training Pengujian Depth dan Width..... 17
Gambar 3.2.1 Grafik Loss Pengujian Pengaruh Fungsi Aktivasi..... 17
Gambar 3.2.2 Grafik Persebaran Bobot..... 18
Gambar 3.2.3 Hasil Pengujian Pengaruh Fungsi Aktivasi..... 19
Gambar 3.3.1 Grafik Loss Pengujian Learning Rate..... 19
Gambar 3.3.2 Grafik Distribusi Bobot..... 20
Gambar 3.3.2 Hasil Uji Pengaruh Learning Rate.....21
Gambar 3.4.1 Grafik Loss Pengujian Pengaruh Regularisasi..... 21
Gambar 3.4.2 Grafik Distribusi Bobot Pengujian Pengaruh Regularisasi.....22
Gambar 3.4.2 Hasil Uji Pengujian Pengaruh Regularisasi..... 23
Gambar 3.5.1 Grafik Loss Perbandingan Inisialisasi Bobot.....23
Gambar 3.5.2 Distribusi Bobot Perbandingan Inisialisasi Bobot.....24
Gambar 3.5.1 Hasil Perbandingan Inisialisasi Bobot.....24
Gambar 3.6.1 Grafik Loss Pengujian Pengaruh Normalisasi RMSNorm.....24
Gambar 3.6.2 Grafik Loss Pengujian Pengaruh Normalisasi RMSNorm.....25
Gambar 3.6.2 Hasil Uji Pengaruh Normalisasi RMSNorm.....26
Gambar 3.7.1 Grafik Loss Pengujian Kecepatan Konvergensi Adam vs GD..... 26
Gambar 3.7.2 Hasil Uji Perbandingan Optimizer..... 26
Gambar 3.8.2 Hasil Uji Perbandingan Optimizer..... 27

01 - Deskripsi Persoalan

Dalam bidang pembelajaran mesin, model prediksi sering digunakan untuk menganalisis hubungan antara berbagai faktor dan suatu hasil tertentu. Salah satu pendekatan yang umum digunakan adalah Feedforward Neural Network (FFNN), yaitu model Neural Network tiruan yang terdiri dari beberapa lapisan neuron yang saling terhubung dan memproses informasi secara berurutan dari lapisan input hingga lapisan output. Pada tugas ini, permasalahan yang diangkat adalah bagaimana merancang dan mengimplementasikan model FFNN dari awal (from scratch) tanpa menggunakan library deep learning siap pakai. Model yang dibangun harus mampu melakukan proses pembelajaran melalui mekanisme forward propagation untuk menghasilkan prediksi serta backward propagation untuk menghitung gradien menggunakan konsep chain rule sehingga bobot jaringan dapat diperbarui menggunakan metode gradient descent.

Selain implementasi model, permasalahan yang diselesaikan juga mencakup penerapan model tersebut pada suatu dataset Internship Placement, yang berisi informasi mengenai performa akademik mahasiswa serta berbagai faktor yang dapat memengaruhi keberhasilan mereka dalam memperoleh internship. Dataset ini mencakup berbagai fitur seperti nilai akademik (CGPA), jumlah mata kuliah yang belum lulus, jumlah internship, kemampuan komunikasi, serta karakteristik institusi pendidikan. Tujuan dari pemodelan ini adalah untuk memprediksi status penempatan kerja (placement status) mahasiswa berdasarkan fitur-fitur tersebut. Dengan memanfaatkan FFNN yang diimplementasikan, diharapkan model dapat mempelajari pola hubungan antara atribut mahasiswa dengan peluang mereka mendapatkan pekerjaan.

Lebih lanjut, permasalahan dalam tugas ini juga melibatkan analisis terhadap berbagai faktor yang memengaruhi performa model, seperti jumlah layer (depth), jumlah neuron pada tiap layer (width), jenis fungsi aktivasi, nilai learning rate, serta penggunaan teknik regularisasi. Analisis ini bertujuan untuk memahami bagaimana perubahan hyperparameter dan konfigurasi arsitektur jaringan dapat memengaruhi proses pembelajaran serta kualitas prediksi model. Hasil yang diperoleh kemudian dibandingkan dengan model Neural Network yang diimplementasikan menggunakan library standar untuk mengevaluasi efektivitas implementasi FFNN yang telah dibuat. Dengan demikian, tugas ini tidak hanya berfokus pada implementasi teknis Neural Network, tetapi juga pada pemahaman mengenai perilaku model dan pengaruh berbagai parameter terhadap performa pembelajaran mesin.

02- Pembahasan

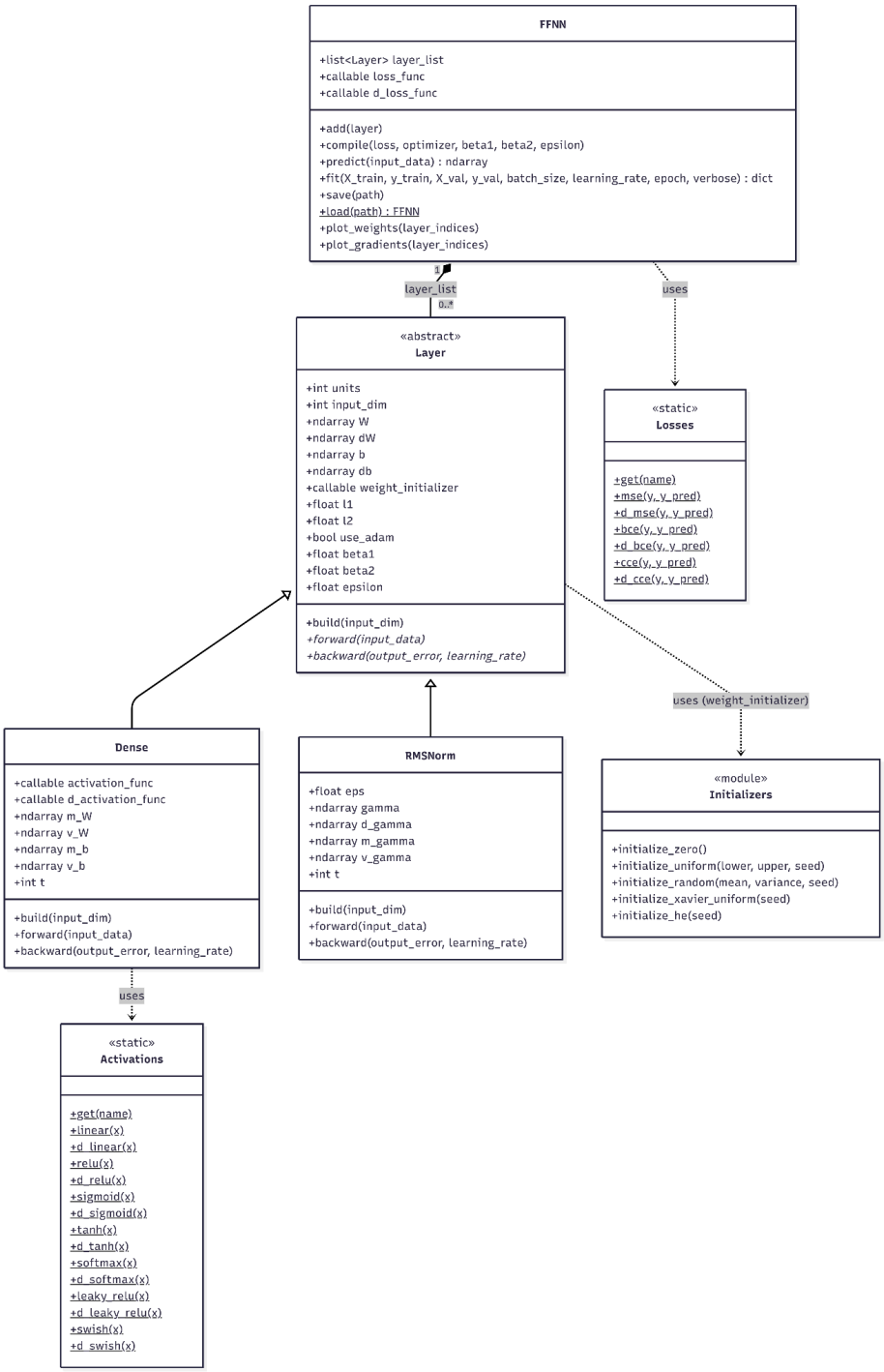
1. Struktur Kode

```
├── README.md
├── data
│   └── datasetml_2026.csv
├── doc
├── main.py
├── notebook
│   └── notebook.ipynb
├── pyproject.toml
├── src
│   ├── __init__.py
│   ├── activations.py
│   ├── initializers.py
│   ├── layers
│   │   ├── __init__.py
│   │   ├── dense.py
│   │   ├── layer.py
│   │   └── rmsnorm.py
│   ├── losses.py
│   ├── model.py
│   └── utils
│       ├── __init__.py
│       ├── persistence.py
│       └── visualization.py
└── uv.lock
```

Dalam tugas besar ini, kami memecah beberapa fungsi yang independen untuk memudahkan dalam pengerjaan. Hal ini juga dilakukan untuk mencegah adanya *blocking* antar anggota tim. Dalam pengerjaan ini kami menggunakan UV sebagai python package manager untuk memudahkan pengaturan dependensi pada *environment* tiap anggota tim. Secara singkat inti dari model FFNN ini terbagi kedalam 5 bagian yaitu *activation*, *initializer*, *layers, losses*, dan *model* yang tiap bagiannya akan dijelaskan pada bagian selanjutnya.

2. Penjelasan Hasil Implementasi

2.1. Diagram Kelas



Gambar 2.1 Diagram Kelas

2.2. Deskripsi Kelas

Berikut adalah deskripsi kelas beserta deskripsi atribut dan methodnya.

Kelas	Atribut/Method	Deskripsi dan Implementasi
Layer (Abstract)	W	Atribut untuk menyimpan matriks bobot
	b	Atribut untuk menyimpan vektor bias
	dW	Atribut untuk menyimpan gradien bobot
	db	Atribut untuk menyimpan gradien bias
	activation	Atribut untuk menyimpan nama fungsi aktivasi
	weight_initializer	Atribut untuk menyimpan metode inisialisasi
	l1	Atribut untuk menyimpan parameter koefisien regularisasi
	l2	Atribut untuk menyimpan parameter koefisien regularisasi
	build(input_dim)	Method untuk menginisialisasi dimensi matriks bobot dan bias berdasarkan jumlah input layer sebelumnya
Dense	activation_func	Atribut untuk menyimpan fungsi aktivasi
	d_activation_func	Atribut untuk menyimpan turunan fungsi aktivasi
	forward(input_data)	Method untuk menghitung output layer melalui transformasi linear dan penerapan fungsi aktivas
	backward(output_error, learning_rate)	Method untuk menghitung gradien kesalahan terhadap bobot, bias, dan input menggunakan chain rule serta memperbarui bobot tersebut
Activations	get(name)	Method untuk mengambil pasangan fungsi aktivasi dan turunannya berdasarkan nama
	linear(x)	Method untuk implementasi fungsi aktivasi linear
	relu(x)	Method untuk implementasi fungsi aktivasi ReLU
	sigmoid(x)	Method untuk implementasi fungsi aktivasi sigmoid
	tanh(x)	Method untuk implementasi fungsi aktivasi hyperbolic tangent
	softmax(x)	Method untuk implementasi fungsi aktivasi softmax
	d_linear(x)	Method untuk perhitungan turunan pertama fungsi aktivasi linear
	d_relu(x)	Method untuk perhitungan turunan pertama fungsi aktivasi ReLu
	d_sigmoid(x)	Method untuk perhitungan turunan pertama fungsi aktivasi sigmoid
	d_tanh(x)	Method untuk perhitungan turunan pertama fungsi aktivasi

		hyperbolic tangent
	d_softmax(x)	Method untuk perhitungan turunan pertama fungsi aktivasi softmax
Losses	get(name)	Method untuk mengambil pasangan fungsi loss dan turunannya berdasarkan nama
	mse(y, y_pred)	Method Method untuk implementasi fungsi Mean Squared Error
	bce(y, y_pred)	Method untuk implementasi fungsi Binary Cross Entropy
	cce(y, y_pred)	Method untuk implementasi fungsi Categorical Cross Entropy
	d_mse(y, y_pred)	Method untuk perhitungan turunan pertama fungsi Mean Squared Error
	d_bce(y, y_pred)	Method untuk perhitungan turunan pertama fungsi Binary Cross Entropy
	d_cce(y, y_pred)	Method untuk perhitungan turunan pertama fungsi Categorical Cross Entropy
FFNN	layer_list	Atribut untuk menyimpan layer-layer di model
	add(layer)	Method untuk menambahkan layer ke dalam model
	compile(loss)	Method untuk menginisialisasi model dengan menginisialisasi setiap layer dan menentukan fungsi Loss
	predict(input_data)	Method untuk melakukan inferensi untuk memprediksi kelas dengan menghitung output layer melalui forward propagation
	fit(X_train, y_train, X_val, y_val, batch_size, learning_rate, epoch, verbose)	Method untuk melakukan training pada model dengan melakukan backward propagation
	save(path)	Method untuk menyimpan model ke sebuah file di storage
	load(path)	Method untuk memuat model yang tersimpan dari sebuah file di storage
	plot_weights(layer_indices)	Method untuk memvisualisasikan distribusi bobot pada layer-layer tertentu
	plot_gradients(layer_indices)	Method untuk memvisualisasikan distribusi gradien bobot pada layer-layer tertentu
RMSNorm	eps	Atribut untuk menyimpan nilai epsilon (konstanta kecil untuk mencegah pembagian dengan nol)
	gamma	Atribut untuk menyimpan nilai gamma
	d_gamma	Atribut untuk menyimpan nilai gradient loss (turunan fungsi Loss terhadap gamma)

	build(input_dim)	Method untuk membangun layer dengan menginisialisasi nilai-nilai awal dari bobot, gamma, dan parameter lainnya
	forward(input_data)	Method untuk menghitung output layer melalui transformasi linear dan penerapan fungsi aktivasi serta dinormalisasi dengan RMSNorm
	backward(output_error, learning_rate)	Method untuk menghitung gradien kesalahan terhadap bobot, bias, dan input menggunakan chain rule serta memperbarui bobot tersebut serta dinormalisasi dengan RMSNorm

Tabel 2.1 Deskripsi Kelas

2.3. Penjelasan Implementasi

Pada bagian ini akan dijelaskan *flow* dan juga penjelasan secara singkat mengenai cara kerja implementasi kode pada tugas besar kami. Komponen utama project tugas besar ini ada pada file **model.py**. Model akan menjadi kelas yang dipanggil ketika ingin menginisiasi FFNN pada kode. FFNN diimplementasikan dengan kode sebagai berikut

model.py

```
class FFNN:
    def __init__(self):
        self.layer_list: list[Layer] = []
        self.loss_func = None
        self.d_loss_func = None

    def add(self, layer: Layer):
        self.layer_list.append(layer)

    def compile(self, loss: str = 'mse', optimizer: str | None = None,
                beta1: float = 0.9, beta2: float = 0.999, epsilon: float = 1e-8):
        if len(self.layer_list) == 0:
            raise ValueError("Model tidak memiliki layer")

        input_size = self.layer_list[0].input_dim
        if input_size is None:
            raise ValueError("Layer pertama harus memiliki input_dim")

        for layer in self.layer_list:
            layer.build(input_size)
            input_size = layer.units

            if optimizer == 'adam':
                layer.use_adam = True
                layer.beta1 = beta1
                layer.beta2 = beta2
                layer.epsilon = epsilon

        self.loss_func, self.d_loss_func = Losses.get(loss)
```

```python
    def predict(self, input_data: np.ndarray) → np.ndarray:
        output = input_data
        for layer in self.layer_list:
            output = layer.forward(output)
        return output

    def fit(self, X_train, y_train, X_val=None, y_val=None,
            batch_size=32, learning_rate=0.01, epoch=100, verbose=1):
        history = {'train_loss': [], 'val_loss': []}
        n = X_train.shape[0]

        for ep in range(epoch):
            indices = np.random.permutation(n)
            X_sh = X_train[indices]
            y_sh = y_train[indices]

            batch_losses = []
            for start in range(0, n, batch_size):
                X_b = X_sh[start:start + batch_size]
                y_b = y_sh[start:start + batch_size]

                output = self.predict(X_b)
                batch_losses.append(self.loss_func(y_b, output))

                error = self.d_loss_func(y_b, output)
                for layer in reversed(self.layer_list):
                    error = layer.backward(error, learning_rate)

            train_loss = float(np.mean(batch_losses))
            history['train_loss'].append(train_loss)

            val_loss = None
            if X_val is not None and y_val is not None:
                val_pred = self.predict(X_val)
                val_loss = self.loss_func(y_val, val_pred)
                history['val_loss'].append(val_loss)

            if verbose == 1:
                bar_len = 30
                filled  = int(bar_len * (ep + 1) / epoch)
                bar     = '=' * filled + '-' * (bar_len - filled)
                msg     = f"\rEpoch {ep + 1}/{epoch} [{bar}] loss: {train_loss:.4f}"
                if val_loss is not None:
                    msg += f" - val_loss: {val_loss:.4f}"
                end = '\n' if ep + 1 == epoch else ''
                print(msg, end=end, flush=True)

        return history
```

Pada model, ada beberapa fungsi penting yang digunakan dalam melakukan pelatihan, yakni *add*, *compile*, dan *fit*. Fungsi *add* merupakan fungsi yang digunakan untuk

menambahkan tiap layer ke dalam model FFNN. Ketika semua layer sudah ditambahkan, layer tersebut perlu dihubungkan untuk mengetahui dimensi input dan dimensi outputnya. Hal tersebut diselesaikan pada pemanggilan fungsi *compile* yang akan memanggil fungsi *build* pada tiap layer untuk menginisialisasi bobot dan menyesuaikan dimensi input dan output. Selain itu, fungsi *compile* juga akan menentukkan fungsi loss pada prediksi di output layer yang nilainya akan digunakan pada chain rule nantinya.

Ketika susunan layer sudah baik dan model sudah dilakukan *compile*, model dapat melakukan *fitting* pada data yang menjadi inputnya. Sederhananya, fungsi *fit* akan melakukan prediksi pada setiap layer yang ada pada model lalu hasil prediksi tersebut akan dihitung errornya dengan memanggil fungsi *backward* yang ada pada masing-masing layer.

Agar lebih jelas mengenai implementasi *forward* dan *backward* yang ada pada layer, berikut penjelasan implementasi pada kode tersebut.

layer.py

```
class Layer(ABC):
    def __init__ (self, units, input_dim = None, activation = None, weight_initializer =
initialize_he(42), l1=0.0, l2=0.0,use_adam=False, beta1=0.9, beta2=0.999, epsilon=1e-8):
        self.input_dim = input_dim
        self.units = units
        self.W = None
        self.dW = None
        self.b = None
        self.db = None
        self.activation = activation
        self.weight_initializer = weight_initializer
        self.l1 = l1
        self.l2 = l2
        self.use_adam = use_adam
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon

    def build(self, input_dim):
        self.input_dim = input_dim
        self.W = self.weight_initializer((input_dim, self.units))
        self.b = np.zeros((1, self.units))

    @abstractmethod
    def forward(self, input_data):
        pass

    @abstractmethod
    def backward(self, output_error, learning_rate):
        pass
```

Pada file ini, kelas abstrak layer diinisialisasi sebagai *blueprint* bagi kelas layer turunan lainnya (Dense dan RMSNorm). Secara umum layer memiliki bobot dan bias yang dimensinya disesuaikan pada pemanggilan fungsi *build*.

dense.py

```
class Dense(Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(units, **kwargs)
        self.activation_func = None
        self.d_activation_func = None
        if activation:
            self.activation_func, self.d_activation_func = Activations.get(activation)
        # atribut untuk Adam optimizer, nanti di-initialize pakai method build()
        self.m_W = None
        self.v_W = None
        self.m_b = None
        self.v_b = None
        self.t = 0

    def build(self, input_dim):
        self.input_dim = input_dim
        self.W = self.weight_initializer((input_dim, self.units))
        self.b = np.zeros((1, self.units))
        # Initialize atribut untuk Adam
        self.m_W = np.zeros((input_dim, self.units))
        self.v_W = np.zeros((input_dim, self.units))
        self.m_b = np.zeros((1, self.units))
        self.v_b = np.zeros((1, self.units))
        self.t = 0

    def forward(self, input_data):
        self.input = input_data
        self.z = np.dot(self.input, self.W) + self.b
        if self.activation_func:
            self.output = self.activation_func(self.z)
        else:
            self.output = self.z
        return self.output

    def backward(self, output_error, learning_rate):
        # hitung Delta (dL/dZ) : dL/dA * dA/dZ
        if self.d_activation_func:
            d_act = self.d_activation_func(self.z)
            if d_act.ndim == 3:
                delta = np.einsum('bi,bij→bj', output_error, d_act)
            else:
                delta = output_error * d_act
        else:
            delta = output_error
```

```
        self.dW = np.dot(self.input.T, delta)
        if self.l1 > 0:
            self.dW += self.l1 * np.sign(self.W)
        if self.l2 > 0:
            self.dW += self.l2 * self.W

        self.db = np.sum(delta, axis=0, keepdims=True)

        input_error = np.dot(delta, self.W.T)

        if self.use_adam:
            self.t += 1
            self.m_W = self.beta1 * self.m_W + (1 - self.beta1) * self.dW
            self.v_W = self.beta2 * self.v_W + (1 - self.beta2) * (self.dW ** 2)
            m_W_hat = self.m_W / (1 - self.beta1 ** self.t)
            v_W_hat = self.v_W / (1 - self.beta2 ** self.t)
            self.W -= learning_rate * m_W_hat / (np.sqrt(v_W_hat) + self.epsilon)

            self.m_b = self.beta1 * self.m_b + (1 - self.beta1) * self.db
            self.v_b = self.beta2 * self.v_b + (1 - self.beta2) * (self.db ** 2)
            m_b_hat = self.m_b / (1 - self.beta1 ** self.t)
            v_b_hat = self.v_b / (1 - self.beta2 ** self.t)
            self.b -= learning_rate * m_b_hat / (np.sqrt(v_b_hat) + self.epsilon)
        else:
            self.W -= learning_rate * self.dW
            self.b -= learning_rate * self.db

        return input_error
```

Dense layer merupakan implementasi dari *fully connected layer*, di mana setiap neuron terhubung ke seluruh neuron pada layer sebelumnya. Pada fungsi *forward*, layer menghitung hasil perkalian matriks input dengan bobot ditambah bias (z = input · W + b), lalu hasilnya dilewatkan ke fungsi aktivasi untuk menghasilkan output layer.

Pada fungsi *backward*, layer menghitung gradien loss terhadap bobot (dW), bias (db), dan input (*input_error*) menggunakan *chain rule*. Gradien terhadap bobot dihitung dari `np.dot(self.input.T, delta)`, di mana delta adalah dL/dZ yang merupakan hasil perkalian *output_error* dengan turunan fungsi aktivasi. Jika regularisasi L1 atau L2 aktif, gradien bobot juga ditambah dengan suku penalti masing-masing. *input_error* kemudian diteruskan ke layer sebelumnya untuk melanjutkan proses *backpropagation*.

Apabila optimizer Adam diaktifkan, pembaruan bobot tidak langsung menggunakan gradien mentah, melainkan melalui estimasi momen pertama (m) dan momen kedua (v) yang diperbarui di setiap langkah. Kedua momen ini kemudian dikoreksi bias-nya (m_hat, v_hat) karena di awal pelatihan nilainya cenderung mendekati nol. Bobot akhirnya diperbarui dengan membagi momen pertama yang terkoreksi dengan akar momen kedua

yang terkoreksi, sehingga learning rate efektif untuk setiap parameter menjadi adaptif dan lebih stabil dibanding SGD biasa.

Selain dense, kami juga mengimplementasikan RMSNorm sebagai layer normalisasi. RMSNorm akan menormalisasi menggunakan Root Mean Square (RMS) dari input. Layer ini memiliki satu parameter yang dapat dilatih, yaitu gamma, sebagai skala hasil normalisasi.

rmsnorm.py

```
class RMSNorm(Layer):
    def __init__(self, units, eps=1e-8, **kwargs):
        kwargs.pop('activation', None)
        super().__init__(units, **kwargs)
        self.eps = eps
        self.gamma = None
        self.d_gamma = None
        # Atribut untuk adam optimizer (di-inisialisasi di method build)
        self.m_gamma = None
        self.v_gamma = None
        self.t = None

    def build(self, input_dim):
        self.input_dim = input_dim
        self.W = np.zeros((input_dim, self.units))
        self.b = np.zeros((1, self.units))
        self.gamma = np.ones((1, self.units))
        self.dW = None
        self.db = None
        self.d_gamma = None
        # Initialize atribut untuk Adam
        self.m_gamma = np.zeros((1, self.units))
        self.v_gamma = np.zeros((1, self.units))
        self.t = 0

    def forward(self, input_data):
        self.input = input_data
        rms = np.sqrt(np.mean(input_data ** 2, axis=-1, keepdims=True) + self.eps)
        self.rms = rms
        self.x_norm = input_data / rms
        self.output = self.gamma * self.x_norm
        return self.output

    def backward(self, output_error, learning_rate):
        n = self.input.shape[-1]

        self.d_gamma = np.sum(output_error * self.x_norm, axis=0, keepdims=True)

        dx_norm = output_error * self.gamma
        drms = np.sum(dx_norm * self.input, axis=-1, keepdims=True) * (-1 / self.rms **
```

```
2)

        input_error = dx_norm / self.rms + drms * (self.input / (n * self.rms))

        if self.use_adam:
            self.t += 1
            self.m_gamma = self.beta1 * self.m_gamma + (1 - self.beta1) * self.d_gamma
            self.v_gamma = self.beta2 * self.v_gamma + (1 - self.beta2) * (self.d_gamma
** 2)
            m_gamma_hat = self.m_gamma / (1 - self.beta1 ** self.t)
            v_gamma_hat = self.v_gamma / (1 - self.beta2 ** self.t)
            self.gamma -= learning_rate * m_gamma_hat / (np.sqrt(v_gamma_hat) +
self.epsilon)
        else:
            self.gamma -= learning_rate * self.d_gamma

        return input_error
```

Pada fungsi *forward*, RMSNorm menghitung nilai RMS dari input pada dimensi terakhir, lalu input dibagi dengan nilai tersebut untuk menghasilkan x_norm. Output akhir diperoleh dari perkalian gamma dengan x_norm, di mana gamma berperan sebagai faktor skala yang dipelajari selama pelatihan.

Pada fungsi *backward*, gradien terhadap gamma dihitung dengan menjumlahkan perkalian elemen antara output_error dan x_norm pada seluruh sampel dalam batch. Selanjutnya, gradien terhadap input (input_error) dihitung dengan mempertimbangkan dua jalur, kontribusi langsung dari normalisasi dan kontribusi tidak langsung melalui perubahan nilai RMS itu sendiri. Penggabungan keduanya memastikan gradien yang diteruskan ke layer sebelumnya sudah memperhitungkan ketergantungan input terhadap nilai RMS-nya sendiri. Pembaruan gamma menggunakan mekanisme yang sama dengan Dense layer, yaitu SGD biasa atau Adam optimizer apabila diaktifkan.

2.4. Forward Propagation

Forward Propagation adalah proses pemindahan data input melalui setiap layer model untuk menghasilkan output. Pada program ini, proses dimulai saat Dense menerima input data dan melakukan transformasi linear menggunakan operasi perkalian matriks dengan bobot dan bias. Hasil operasi tersebut kemudian diproses melalui fungsi aktivasi yang dipilih saat melakukan inisialisasi. Hasil akhir fungsi aktivasi tersebut kemudian disimpan sebagai output dan diteruskan sebagai input untuk layer berikutnya.

2.5. Backward Propagation

Backward Propagation adalah proses yang digunakan untuk mengoptimalkan bobot pada *artificial neural network* dengan cara mengukur kesalahan yang terjadi pada setiap lapisan jaringan dan menyesuaikan bobot sesuai dengan kesalahan tersebut. Pada

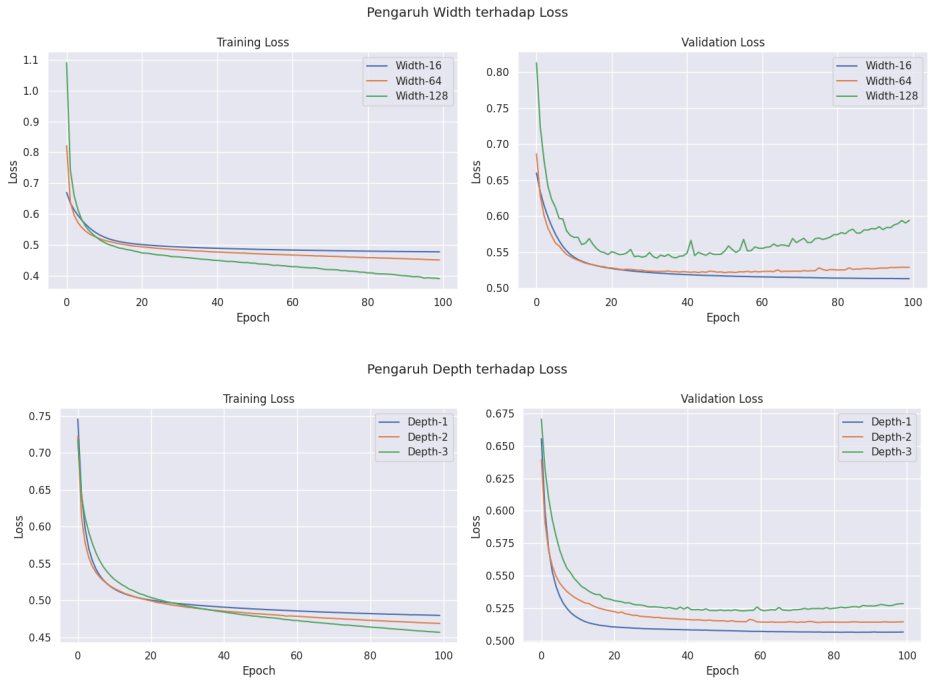
program ini, proses ini diawali dengan menghitung turunan fungsi loss sebagai representasi selisih antara prediksi dengan target asli. Di dalam layer Dense, turunan fungsi loss tadi dikalikan dengan turunan fungsi aktivasi terhadap nilai Z untuk mendapatkan nilai delta. Kemudian, model menghitung gradien bobot (dW) melalui perkalian matriks antara transpose input dengan delta, serta menghitung gradien bias dengan menjumlahkan nilai delta.

Jika konfigurasi regularisasi L1 atau L2 diaktifkan pada layer, penalti tambahan akan ditambahkan langsung ke dW untuk mengontrol besaran bobot. Proses terakhir adalah *weight update.* Dalam proses ini, bobot dan bias diperbarui dengan cara mengalikan gradien yang telah disimpan dengan *learning rate.* Nilai bobot dan bias lama kemudian dikurangi dengan hasil perkalian tersebut.

3. Hasil Pengujian

3.1. Pengaruh Depth dan Width

Dilakukan pengujian untuk mengetahui pengaruh ukuran depth dan width pada model FFNN. Dalam dua tahap yaitu Width Variation (depth = 2 hidden layers) dan Depth Variation (width = 32 per hidden layer). Hasilnya adalah sebagai berikut:



Gambar 3.1.1 Grafik Loss Pengujian Depth dan Width

Dari hasil pengujian terlihat bahwa width yang semakin besar dapat membuat model semakin overfit. Hal ini ditandai dengan training loss yang turun jauh tetapi validation loss yang naik signifikan. Hal yang sama juga terjadi pada depth. Jika jumlah layer semakin

banyak, model akan semakin overfit. Maka dari itu, perlu dicari nilai yang ideal agar tidak overfit.

Config	Accuracy	F1 Score
Width-16	0.7505	0.8032
Width-64	0.7465	0.7998
Width-128	0.7195	0.7760

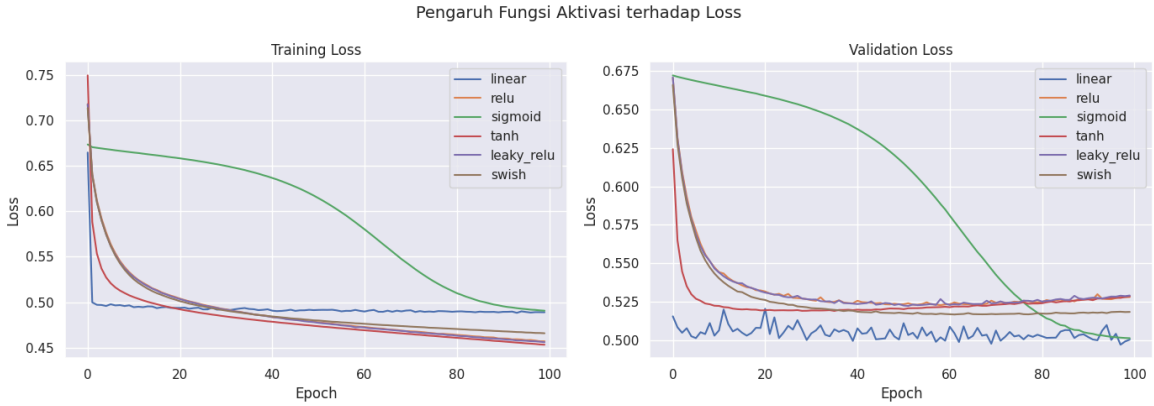
Config	Accuracy	F1 Score
Depth-1	0.745	0.8014
Depth-2	0.745	0.8012
Depth-3	0.737	0.7937

Gambar 3.1.2 Hasil Training Pengujian Depth dan Width

Nilai akurasi dan F1 Score juga ikut memperkuat temuan ini. Model yang memiliki width ataupun depth terlalu besar akan overfit sehingga skor dan akurasinya berkurang.

3.2. Pengaruh Fungsi Aktivasi

Dilakukan pengujian untuk mengetahui pengaruh fungsi aktivasi terhadap model. Dilakukan pengujian dengan fungsi aktivasi linear, ReLu, sigmoid, tanh, leaky_relu, dan swish.



Gambar 3.2.1 Grafik Loss Pengujian Pengaruh Fungsi Aktivasi

Dari pengujian, terlihat bahwa bahwa rata-rata semua fungsi aktivasi menurunkan training loss dengan baik kecuali fungsi sigmoid yang mengalami penurunan paling lambat dan fungsi linear yang *plateu*/mendatar setelah tiba-tiba turun drastis di awal. Untuk validation loss, hampir semua fungsi aktivasi turun dengan baik kecuali sigmoid yang lambat dan fungsi linear yang tidak stabil dan fluktuatif.

Gambar 3.2.2 Grafik Persebaran Bobot

Dari perbandingan distribusi pada beberapa layer. Terlihat bahwa persebaran bobot cukup merata (tidak terfokus pada nilai 0) dan simetris (meskipun tidak terlalu sempurna dan tidak terlalu menyerupai distribusi normal). Sedangkan untuk persebaran gradien, dengan fungsi aktivasi ReLU, Leaky ReLU, dan Swish, pada layer 0 dan layer 1, gradien terlihat normal dan terdistribusi dengan baik. Namun, pada layer 2, terdapat di angka 0 sumbu X, terjadi lonjakan sangat tinggi. Ini menunjukkan banyaknya neuron yang berhenti belajar. Sedangkan untuk tanh dan fungsi linear, kedua fungsi aktivasi ini menunjukkan distribusi gradien yang stabil di semua layer. Rentang nilai gradien konsisten di kisaran -0.04 hingga 0.06. Sebaliknya, fungsi sigmoid gagal menjaga kestabilan rentang gradien. Pada fungsi aktivasi sigmoid, sumbu-x pada grafik nilainya jauh lebih kecil dibandingkan aktivasi lainnya dan terus menyusut. Ini menunjukkan bahwa learning melambat.

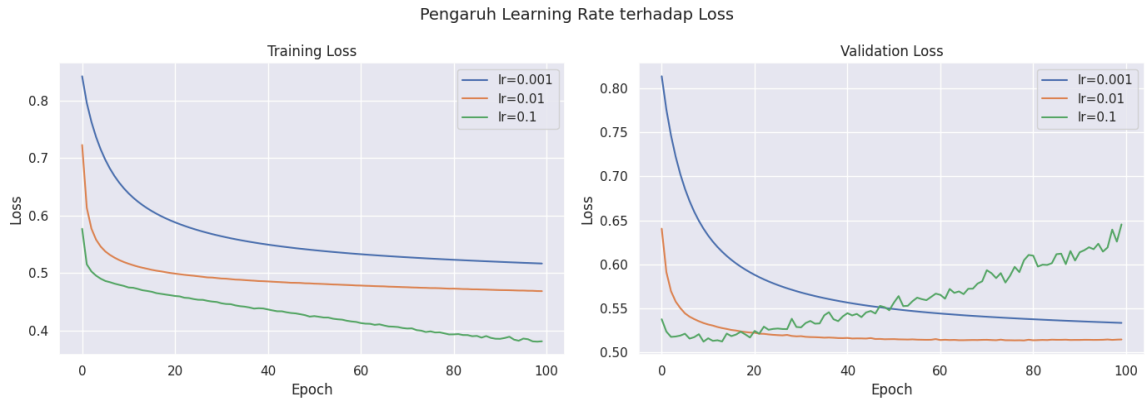
Activation	Accuracy	F1 Score
linear	0.7560	0.8101
relu	0.7385	0.7951
sigmoid	0.7550	0.8081
tanh	0.7415	0.7970
leaky_relu	0.7380	0.7930
swish	0.7415	0.7975

Gambar 3.2.3 Hasil Pengujian Pengaruh Fungsi Aktivasi

Dari hasil pengujian, terlihat bahwa fungsi aktivasi linear tetap bisa mendapatkan nilai paling baik dengan akurasi paling tinggi meskipun grafik lossnya cukup buruk. Fungsi aktivasi dengan skor dan akurasi terbaik kedua adalah fungsi sigmoid yang grafik lossnya juga buruk dan learningnya yang lambat.

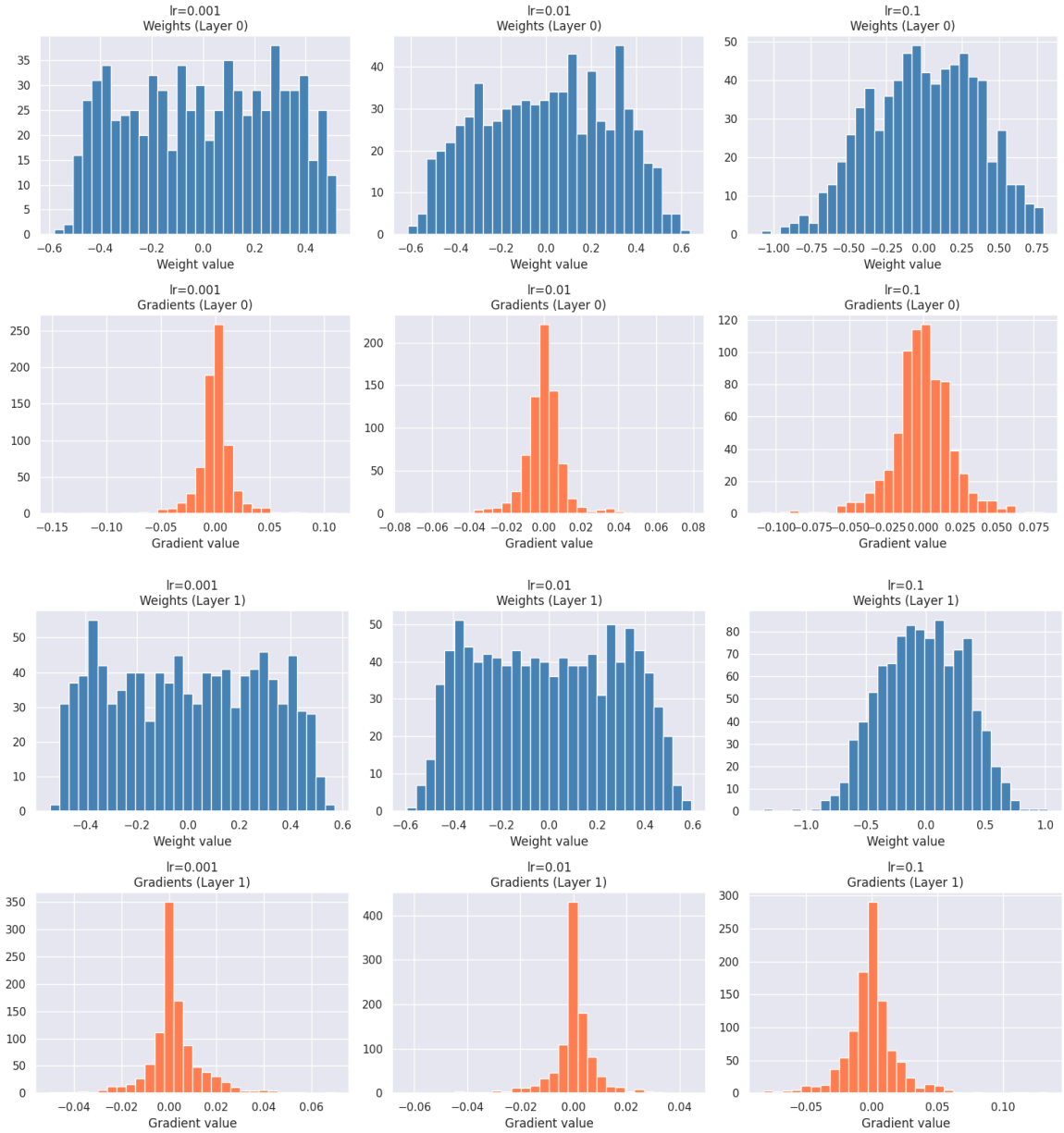
3.3. Pengaruh Learning Rate

Untuk mengetahui pengaruh learning rate, dilakukan pengujian dengan 3 value learning rate berbeda, yaitu 0.001, 0.01, dan 0.1. Berikut adalah hasilnya



Gambar 3.3.1 Grafik Loss Pengujian Learning Rate

Dari grafik loss terlihat bahwa learning rate yang semakin terlalu tinggi cenderung membuat model menjadi overfit. Meski begitu, jika nilai learning rate cukup tinggi tapi tidak terlalu tinggi maka ia bisa menyebabkan grafik training loss dan validation loss model turun dengan sangat baik dan smooth.



Gambar 3.3.2 Grafik Distribusi Bobot

Sedangkan untuk analisis distribusi weight dan gradien tiap layer, terlihat bahwa pada layer kedua semua model cenderung menjalankan training yang sangat lambat (banyak nilai gradien nol). Namun, pada layer pertama, model dengan learning rate paling besar cenderung memiliki gradien yang tersebar lebih merata (tidak terlalu melonjak di angka 0) sehingga learning masih berjalan dengan cepat di sini. Pada persebaran bobot juga terlihat bahwa model dengan learning rate 0.1 cenderung lebih lebar distribusi bobotnya (dari -1.4 hingga 1.0)

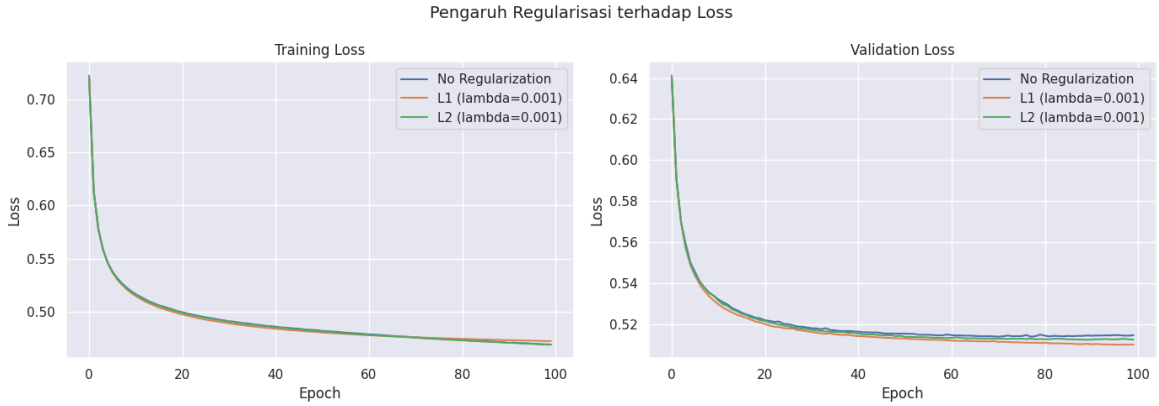
Learning Rate	Accuracy	F1 Score
lr=0.001	0.7305	0.7917
lr=0.01	0.7435	0.7991
lr=0.1	0.7150	0.7834

Gambar 3.3.2 Hasil Uji Pengaruh Learning Rate

Ketika melihat skor dan akurasinya, semakin jelas bahwa model dengan learning rate yang pas, tidak terlalu rendah dan tidak terlalu tinggi menjadi lebih akurat dan lebih bagus performanya.

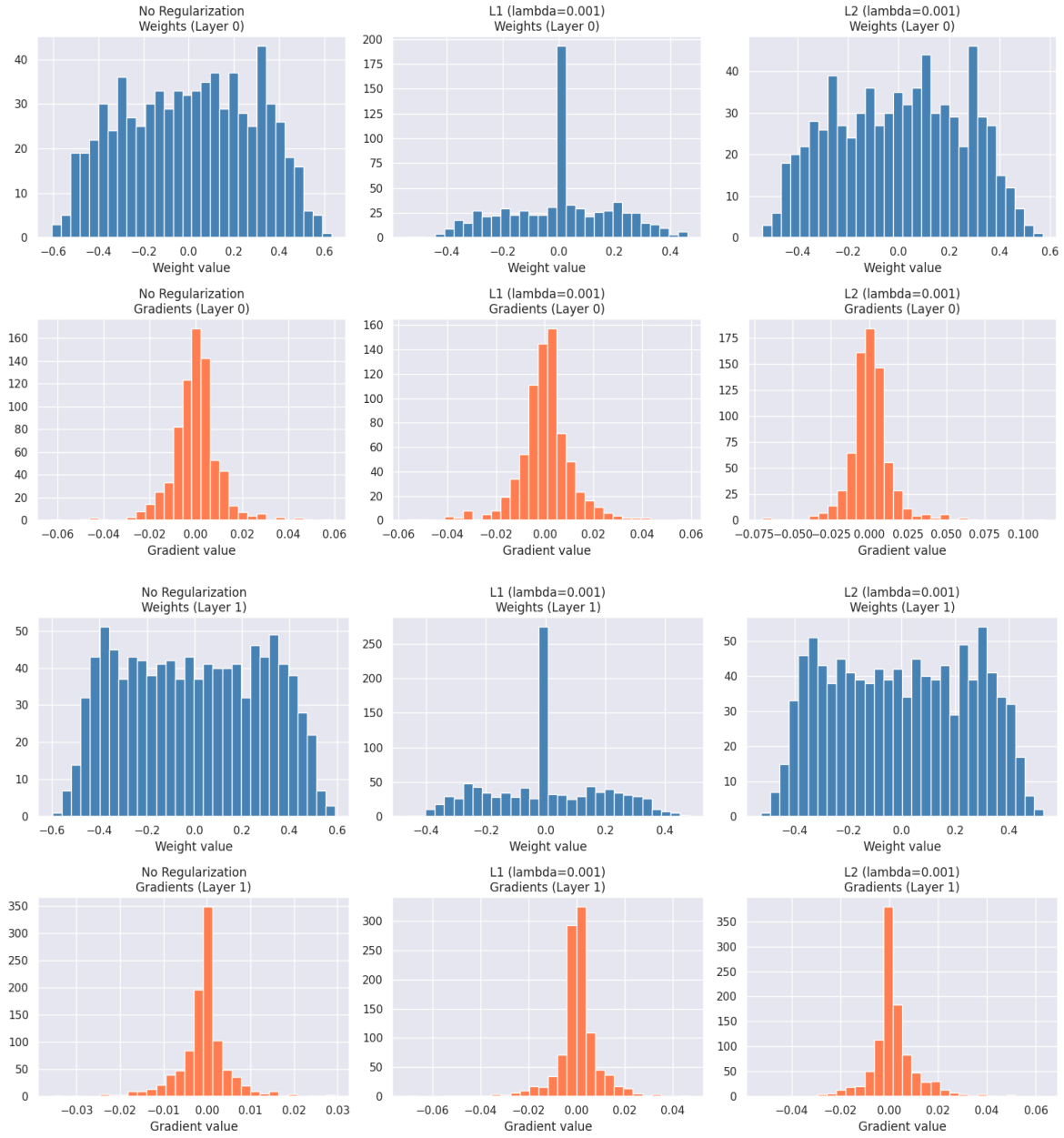
3.4. Pengaruh Regularisasi

Dilakukan training tanpa regularisasi, dengan L1, dan dengan L2. Hasilnya adalah sebagai berikut.



Gambar 3.4.1 Grafik Loss Pengujian Pengaruh Regularisasi

Dari grafik, tidak terlihat adanya perbedaan training loss dan validation loss pada model yang menggunakan regularisasi dan tidak menggunakan regularisasi.



Gambar 3.4.2 Grafik Distribusi Bobot Pengujian Pengaruh Regularisasi

Pada layer kedua, terlihat bahwa distribusi bobot cenderung menjadi lebih simetris. Sedangkan pada distribusi gradien, model dengan L2 dan model tanpa regularisasi sama-sama membuat learning menjadi lebih lambat karena banyaknya gradien di angka 0. Sedangkan untuk L1 juga melambat tetapi tidak seburuk L2 dan tanpa regularisasi.

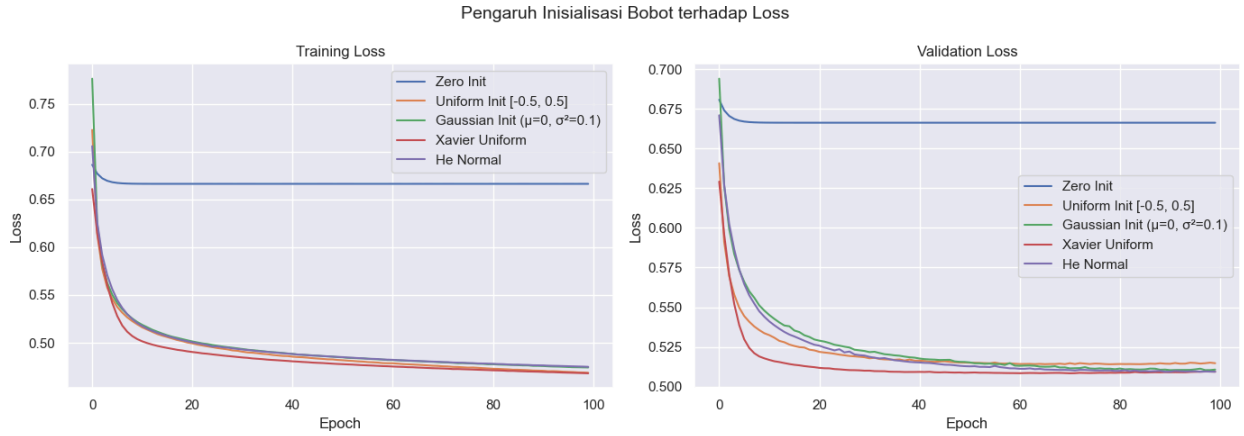
Regularization	Accuracy	F1 Score
No Regularization	0.7465	0.8034
L1 (lambda=0.001)	0.7475	0.8025
L2 (lambda=0.001)	0.7480	0.8045

Gambar 3.4.2 Hasil Uji Pengujian Pengaruh Regularisasi

Akurasi dan skor yang didapatkan model dengan L2 paling tinggi, model tanpa regularisasi ada di urutan kedua, kemudian model dengan L1 performanya paling buruk tetapi akurasinya terbaik kedua. Perbedaan ketiganya tidak terlalu signifikan.

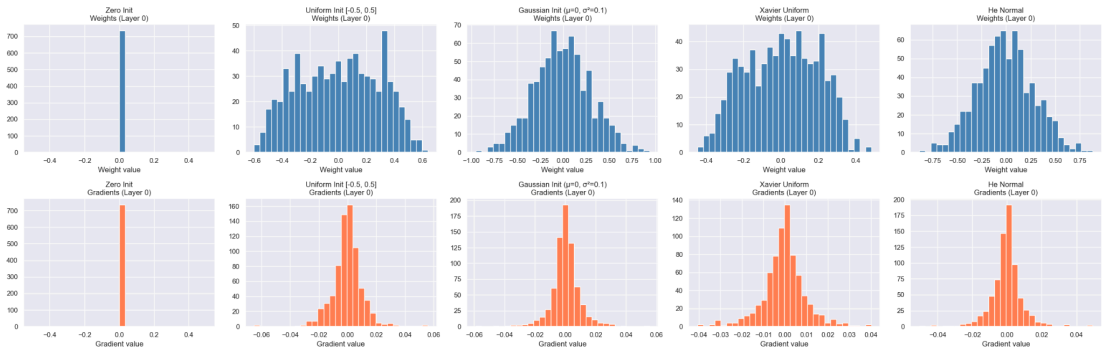
3.5. Pengaruh Inisialisasi Bobot

Pada percobaan ini, diuji pula pengaruh inisialisasi bobot pada berbagai model yang kami uji



Gambar 3.5.1 Grafik Loss Perbandingan Inisialisasi Bobot

Pada perbandingan ini terlihat secara signifikan nilai loss terhadap jenis inisialisasi bobot yang digunakan terutama pada *zero initialization*. Ketika menggunakan inisialisasi tersebut nilai loss cenderung tidak mengalami pengurangan yang signifikan tiap epoch-nya dan stagnan di angka sekitar 0.675 - 0.625, sangat berbanding jauh dengan jenis inisialisasi lain yang berada dibawah 0.525 .Hal ini terjadi karena jika semua bobot neuron dibuat sama, maka perubahan bobot pada tiap neuron di satu layer tersebut akan sama pula dan setiap output yang dihasilkan juga sama untuk setiap neuron, hal ini membuat satu semua unit neuron pada satu layer tersebut seolah berperilaku sebagai satu neuron tunggal, sehingga kapasitas model untuk mempelajari representasi yang beragam menjadi sangat terbatas. Kondisi ini dikenal sebagai *symmetry breaking problem*, di mana model gagal memecah simetri antar neuron sejak awal pelatihan dan menyebabkan proses pembelajaran menjadi tidak efektif meskipun jumlah neuron pada layer tersebut banyak.



Gambar 3.5.2 Distribusi Bobot Perbandingan Inisialisasi Bobot

Hal ini terlihat jelas pada grafik distribusi bobot. Pada model yang menggunakan inisialisasi selain *zero initialization* bobot tersebut memiliki nilai dalam rentang tertentu berdasarkan formula fungsi masing-masing.

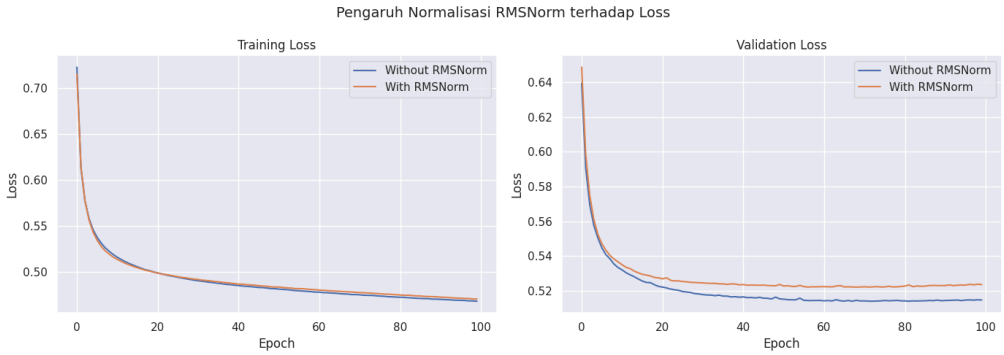
Initializer	Accuracy	F1 Score
Zero Init	0.6155	0.7620
Uniform Init [-0.5, 0.5]	0.7455	0.8025
Gaussian Init (μ=0, σ^2=0.1)	0.7490	0.8019
Xavier Uniform	0.7505	0.8059
He Normal	0.7525	0.8070

Gambar 3.5.1 Hasil Perbandingan Inisialisasi Bobot

Pada hasil akhir juga tervalidasi bahwa model dengan *zero initialization* memiliki performa yang jauh lebih rendah dibandingkan dengan performa inisialisasi bobot yang lain.

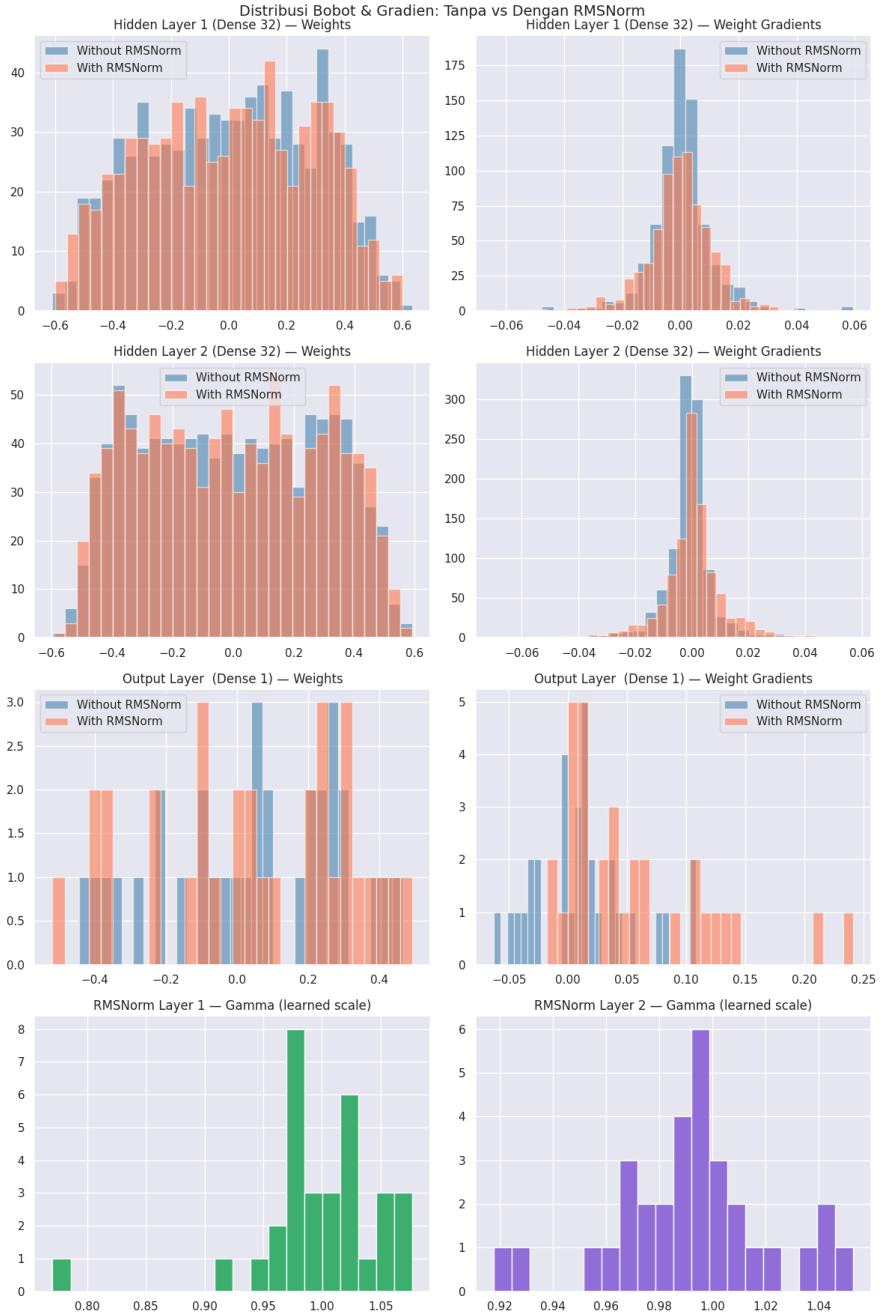
3.6. Pengaruh Normalisasi RMSNorm

Dilakukan training model dengan dan tanpa normalisasi RMSNorm. Hasilnya adalah sebagai berikut.



Gambar 3.6.1 Grafik Loss Pengujian Pengaruh Normalisasi RMSNorm

Terlihat bahwa validation loss model dengan RMSNorm hasilnya lebih baik.



Gambar 3.6.2 Grafik Loss Pengujian Pengaruh Normalisasi RMSNorm

Dari hasil di atas, distribusi tidak terlalu berbeda. Namun, pada distribusi gradien, terlihat bahwa di kedua hidden layer, model dengan RMSNorm proses pelatihannya lebih cepat (gradien tidak terlalu melonjak di 0). Model dengan RMSNorm juga tidak terlalu melambat di layer kedua, tidak seperti model tanpa RMSNorm.

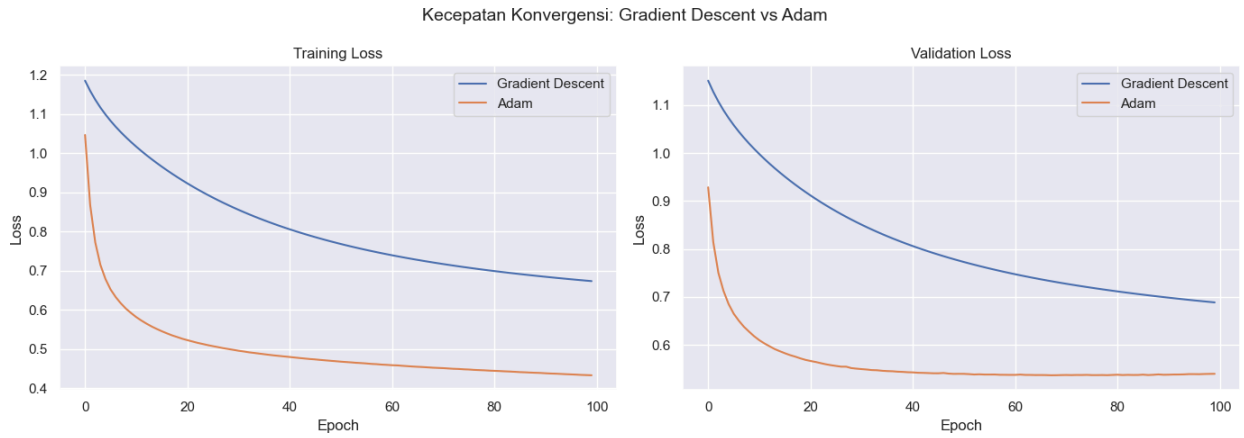
Model	Accuracy	F1 Score
Without RMSNorm	0.7435	0.8000
With RMSNorm	0.7440	0.7983

Gambar 3.6.2 Hasil Uji Pengaruh Normalisasi RMSNorm

Pada perhitungan akurasi dan skor F1, hasil kedua model tidak terlalu berbeda dan tidak terlalu berarti.

3.7. Kecepatan Konvergensi Adam vs Gradient Descent

Dilakukan training untuk melatih perbedaan kecepatan konvergensi optimizer Adam dengan optimizer SGD.



Gambar 3.7.1 Grafik Loss Pengujian Kecepatan Konvergensi Adam vs GD

Dalam pengujian menggunakan 100 epoch, model yang menggunakan optimizer Adam lebih cepat mengalami konvergensi dibanding dengan menggunakan Gradient Descent. Dalam hal ini model dengan optimizer Adam mampu menyentuh 90% konvergensi pada epoch ke 18 sedangkan Gradient Descent pada epoch ke 68.

Optimizer	Final Val Loss	Best Val Loss	Best Epoch	Epoch 90% Converged	Total Time (s)	Time / Epoch (ms)	Accuracy	F1 Score
Adam	0.53939	0.53635	68	18	20.089	200.889	0.735	0.7900
Gradient Descent	0.68841	0.68841	100	68	14.558	145.575	0.677	0.7399

Gambar 3.7.2 Hasil Uji Perbandingan Optimizer

Walaupun secara nilai epoch Adam lebih cepat untuk mencapai kondisi konvergen, waktu yang diperlukan untuk melakukan satu epoch lebih lambat 38% (55ms) dibandingkan dengan waktu pada Gradient Descent. Namun, dengan pengujian total 100 epoch, model dengan optimizer Adam dapat memberikan metrik F1 Score yang jauh lebih baik dibanding Gradient Descent.

3.8. Perbandingan dengan Library Scikit-Learn

Dilakukan training menggunakan model dari tugas ini dan model MPL dari sklearn dengan hyperparameter yang sama. Hasil akhirnya adalah sebagai berikut.

Model	Accuracy	F1 Score
Our FFNN	0.7500	0.8039
sklearn MLP	0.7485	0.8013

```
--- FFNN Bigmo ---
              precision    recall  f1-score   support

  Not Placed       0.70      0.62      0.66       769
      Placed       0.78      0.83      0.80      1231

    accuracy                           0.75      2000
   macro avg       0.74      0.73      0.73      2000
weighted avg       0.75      0.75      0.75      2000

--- sklearn MLP ---
              precision    recall  f1-score   support

  Not Placed       0.69      0.63      0.66       769
      Placed       0.78      0.82      0.80      1231

    accuracy                           0.75      2000
   macro avg       0.73      0.73      0.73      2000
weighted avg       0.75      0.75      0.75      2000
```

Gambar 3.8.2 Hasil Uji Perbandingan Optimizer

Dari hasil di atas terlihat bahwa performa model kami dengan sklearn MLP tidak beda jauh. Model buatan kami berhasil untuk memiliki performa yang sama bahkan sedikit lebih baik daripada sklearn MLP. Terlihat pada bagian precision memiliki nilai 0.70 pada not placed sedangkan pada sklearn MPL memiliki skor 0.69.

03- Kesimpulan dan Saran

Kesimpulan

Berdasarkan serangkaian pengujian yang telah dilakukan, dapat disimpulkan bahwa pemilihan arsitektur dan hyperparameter memiliki pengaruh yang signifikan terhadap kinerja model Feedforward Neural Network (FFNN). Penggunaan depth dan width yang terlalu besar cenderung menyebabkan overfitting, sehingga diperlukan penentuan nilai yang optimal agar model mampu melakukan generalisasi dengan baik.

Fungsi aktivasi juga memengaruhi kestabilan proses pelatihan. Fungsi sigmoid menunjukkan proses pembelajaran yang paling lambat akibat permasalahan vanishing gradient, sedangkan fungsi linear, meskipun menghasilkan grafik loss yang kurang stabil, mampu memberikan akurasi tertinggi.

Pemilihan learning rate merupakan faktor krusial dalam proses pelatihan. Nilai yang terlalu tinggi dapat menyebabkan *overfitting*, sedangkan nilai yang terlalu rendah memperlambat proses konvergensi. Oleh karena itu, diperlukan pemilihan nilai yang tepat agar diperoleh keseimbangan antara kecepatan dan stabilitas pelatihan.

Dari sisi regularisasi, metode L1 dan L2 tidak menunjukkan perbedaan yang signifikan pada dataset yang digunakan, meskipun L2 sedikit lebih unggul dalam hal performa. Sementara itu, penggunaan *zero initialization* terbukti tidak efektif karena menimbulkan *symmetry breaking problem* yang menghambat proses pembelajaran model.

Penerapan RMSNorm diketahui dapat mempercepat proses pelatihan, khususnya pada jaringan yang lebih dalam, meskipun peningkatan akurasi yang dihasilkan tidak terlalu signifikan. Selain itu, optimizer Adam terbukti lebih cepat mencapai konvergensi dibandingkan dengan SGD.

Secara keseluruhan, model FFNN yang dibangun dari awal mampu menandingi, bahkan sedikit melampaui, performa model MLP dari pustaka scikit-learn pada dataset yang sama.

Saran

Untuk pengembangan lebih lanjut, disarankan dilakukan pengujian menggunakan dataset yang lebih besar dan beragam guna meningkatkan validitas dan generalisasi hasil penelitian.

Selain itu, eksplorasi teknik regularisasi lain, seperti *dropout*, dapat dipertimbangkan untuk mengatasi *overfitting* yang muncul akibat penggunaan *depth* dan *width* yang besar.

Pengujian dengan variasi hyperparameter yang lebih luas juga perlu dilakukan, terutama dalam menemukan kombinasi learning rate, depth, dan width yang paling optimal.

Terakhir, implementasi dan perbandingan teknik normalisasi lain, seperti Batch Normalization, dengan RMSNorm dapat dilakukan untuk menganalisis pengaruhnya terhadap kecepatan konvergensi serta performa akhir model secara lebih komprehensif.

04 - Pembagian Tugas

Nama	NIM	Tugas
Orvin Andika Ikhsan A	13523017	Fungsi Sigmoid, Hyperbolic Tangent, MSE, BCE, Forward Propagation, Backward Propagation
Fajar Kurniawan	13523027	Inisialisasi zero, uniform, random, fungsi aktivasi softmax, CCE, leaky ReLu, Swish
Adhimas Aryo Bimo	13523052	Model FFNN, Fungsi ReLu, Linear, Abstraksi kelas Layers, Fungsi HE, Fungsi Initializer

05 - Referensi

- https://www.jasonosajima.com/forwardprop
- https://www.jasonosajima.com/backprop
- https://www.v7labs.com/blog/neural-networks-activation-functions
- https://arxiv.org/pdf/1412.6980
- https://www.geeksforgeeks.org/deep-learning/kaiming-initialization-in-deep-learning/
- https://www.geeksforgeeks.org/deep-learning/xavier-initialization/
- https://www.geeksforgeeks.org/machine-learning/leaky-relu-activation-function-in-deep-learning/
- https://www.geeksforgeeks.org/deep-learning/swish-activation-function/
- https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/

06 - Lampiran

Format Deklarasi Penggunaan AI
Surat Pernyataan Penggunaan AI

Dengan ini, kami :

1. Orvin Andhika Ikhsan A / 13523017
2. Fajar Kurniawan / 13523027
3. Adhimas Aryo Bimo / 13523052

Fakultas / Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas / Proposal : Feedforward Neural Network – Tugas Besar 1 IF3270 Pembelajaran Mesin

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/ Kecerdasan buatan yang kami gunakan adalah :

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer,	Tidak	1 - No AI

Gemini, Canva AI, atau sejenisnya.		
Pembuatan Kode : menggunakan tools seperti ChatGPT, Gemini, Copilot, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring

Kami juga menyatakan bahwa setiap penggunaan AI/Kecerdasan buatan yang dilakukan pada mata kuliah tersebut dinyatakan di dalam surat ini.

Bandung ,18 Maret 2026

Orvin Andhika Ikhsan A

NIM 13523017